UCS503 – Software Engineering



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

RaftKV

Design, Architecture and Progress of a Dynamic-Membership Distributed Key-Value Store on the Raft Consensus Algorithm

---

Submitted by:

1024240104	Anubhav yadav
1024240003	Kunwar aran srivastava
1024240143	Yuvraj kumar
1024240101	Pratik batish

Group: 3X15 (B.E - 3rd Year, AIML)

Submitted to:

Ms. Anushka

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

October 2026

Abstract

This report presents the design, architecture and implementation progress of **RaftKV**, a fault-tolerant, strongly consistent distributed key-value store written from scratch in Go on top of the Raft consensus algorithm. Unlike a fixed-size cluster, RaftKV is designed for **dynamic membership**: a new node joins a running cluster as a non-voting learner, catches up through log replication or a snapshot, and is then promoted to a voting member through a committed configuration change.

The system separates consensus, state-machine application, durable storage, inter-node transport, client API and membership management behind explicit interfaces, so that each concern can be tested in isolation and under injected faults. Clients use an HTTP/JSON API with leader redirection; nodes communicate over gRPC using RequestVote, AppendEntries, InstallSnapshot and JoinCluster. Durability is provided by a checksummed, fsync'd write-ahead log and periodic snapshots.

The report documents the system from six software-engineering views (use case, class, data flow, sequence, state transition and deployment), states the requirements and safety invariants the implementation must satisfy, and describes the staged delivery plan, effort estimate, risk register and test strategy. At the mid-semester point, Stage 1 (repository, interfaces, protocol and API skeletons, Docker Compose, CI) and Stage 2 (the standalone key-value state engine with deterministic apply, snapshot and restore) are implemented. The consensus algorithm, durable storage and membership are not yet implemented, and no performance results are claimed; the report therefore defines the evaluation plan rather than reporting measurements.

Keywords: Raft consensus, distributed key-value store, dynamic membership, replicated state machine, write-ahead log, fault injection, Go.

Contents

1 Introduction 1
- 1.1 Background and Context 1
- 1.2 Problem Statement 1
- 1.3 Project Objectives 2
- 1.4 Scope 2
- 1.5 Organisation of this Report 2

2 Requirements and System Model 3
- 2.1 Stakeholders and Use Cases 3
- 2.2 Functional Requirements 4
- 2.3 Non-Functional Requirements 4
- 2.4 System Model and Assumptions 5

3 Methodology and Design 6
- 3.1 Development Methodology 6
- 3.2 System Architecture 7
- 3.3 Structural Design 9
- 3.4 Behavioural Design 11
- 3.5 Persistence and Recovery 14
- 3.6 Dynamic Membership 14
- 3.7 Interfaces and Data Structures 16
- 3.8 Deployment Design 16
- 3.9 Correctness Properties 17
- 3.10 Failure Handling 18
- 3.11 Design Evolution and Key Decisions 19

4 Implementation Progress and Project Management 21
- 4.1 Current Implementation Status 21
- 4.2 Plan and Roadmap 22
- 4.3 Tracking with GitHub Projects 23
- 4.4 Effort, Capacity and Cut Line 24
- 4.5 Team Organisation 25
- 4.6 Risk Management 26

5 Testing and Evaluation 27
- 5.1 Test Strategy 27
- 5.2 Fault-Injection Architecture 27
- 5.3 Verification at Mid-Semester 28
- 5.4 Evaluation Plan and Metrics 28
- 5.5 Analysis and Discussion 29

6 Conclusion 30
- 6.1 Summary of Work 30
- 6.2 Key Findings 30
- 6.3 Future Work and Recommendations 30
- 6.4 Final Remarks 30

References 32

Appendix A: HTTP API and CLI 33

Appendix B: Phase Breakdown 34

Appendix C: Reproducing the Current Build 36

Appendix D: Glossary 37

List of Figures

Figure 2.1 Use case diagram: client, cluster operator and peer Raft nodes. 3
Figure 3.1 Layered architecture of a single RaftKV node. 7
Figure 3.2 Module dependency rules: allowed imports, interface implementation and forbidden dependencies. 9
Figure 3.3 UML class diagram of the RaftKV software abstractions (design baseline). 10
Figure 3.4 Raft node state transitions among follower, candidate and leader. 11
Figure 3.5 Data flow of a client operation through the leader, follower logs, persistence and state machine. 12
Figure 3.6 Sequence of a PUT request: replication, majority commit and state-machine application. 13
Figure 3.7 Dynamic join protocol: learner admission, catch-up and promotion to voter. 15
Figure 3.8 Target deployment: one container and one volume per node on a Docker network. 17
Figure 4.1 Delivery status by stage at mid-semester. 21
Figure 4.2 Planned roadmap for Stages 3–8 at phase level (sprints S1–S10). 23
Figure 4.3 Estimated effort by stage and comparison with team capacity. 24

List of Tables

Table 1.1 Scope classification (SDD section 5) 2
Table 2.1 Stakeholders and their interests 3
Table 2.2 Functional requirements and where they are planned 4
Table 2.3 Non-functional requirements and how each is verified 4
Table 2.4 Assumptions of the system model 5
Table 3.1 Engineering practices and their status 6
Table 3.2 Technology choices and rationale 7
Table 3.3 Module responsibilities 8
Table 3.4 Core interfaces in the frozen contracts 10
Table 3.5 Client-facing responses and their effect on the log 13
Table 3.6 Write-ahead log record format 14
Table 3.7 Persisted data structures 16
Table 3.8 Node-to-node RPCs 16
Table 3.9 Node configuration 17
Table 3.10 Raft safety invariants 17
Table 3.11 Failure scenarios and expected behaviour 18
Table 3.12 Evolution from the design baseline to the SDD 19
Table 3.13 Key design decisions 19
Table 4.1 Component status at mid-semester 21
Table 4.2 Stages, schedule and effort 22
Table 4.3 Project board fields and views 24
Table 4.4 Cut line if the schedule slips 25
Table 4.5 Roles, modules and planned load (Stages 3–8) 25
Table 4.6 Risk register 26
Table 5.1 Test tiers 27
Table 5.2 Evaluation metrics and their status 28
Table A.1 HTTP endpoints 33
Table B.1 Phases, owners, windows and effort 34
Table D.1 Terms used in this report 37

1 Introduction

1.1 Background and Context

Modern services keep data on more than one machine so that a hardware failure, a reboot or a network fault neither makes the service unavailable nor loses data. Keeping several copies *consistent* is difficult: if two replicas accept different writes while a network partition separates them, the system ends up with diverging state (“split brain”), and clients may read values that the cluster never agreed on. Consensus algorithms solve this by making a group of machines agree on one ordered sequence of commands. Every replica that applies the same commands in the same order reaches the same state, which is the replicated-state-machine approach [1], [3].

Raft [1] is a consensus algorithm designed to be understandable. It splits the problem into leader election, log replication and safety. Time is divided into *terms*; in each term at most one leader is elected by a majority of voters. The leader receives client commands, appends them to its log and replicates them to followers. An entry is *committed* once it is stored on a majority, and only then applied to the state machine. Because any two majorities of a cluster intersect, a newly elected leader always holds every committed entry.

RaftKV applies Raft to a key-value store. A key-value store is a deliberately small state machine (PUT, GET and DELETE), so the engineering effort goes into the consensus, durability and membership machinery rather than into query semantics. The repository name, *dynamic-raft-kv*, reflects the distinguishing requirement of the project: the cluster must be able to grow at run time through an explicit join protocol in which a new node first joins as a non-voting *learner*, is brought up to date, and is promoted to voter only afterwards.

The central software component is `RaftNode`. It maintains the current term, the vote cast in that term, the replicated log, the commit index, the applied index and, on the leader, the replication progress of every follower. A `StateMachine` interface receives committed entries (implemented by the in-memory key-value engine), a `Storage` interface persists Raft metadata and log entries, and a `Transport` interface abstracts node-to-node RPCs. Chapter 3 describes these relationships in detail.

1.2 Problem Statement

A single-node key-value store has no fault tolerance: a crash or restart loses availability, and without replication a machine failure can lose data. Naively copying writes to several nodes does not help, because without a consensus protocol a partition can produce lost writes, split brain or permanently diverging replicas. RaftKV addresses this by providing a strongly consistent, replicated key-value store that:

- continues to serve writes as long as a majority (quorum) of voting nodes is healthy and reachable;
- refuses to commit writes when a quorum cannot be reached, rather than risking divergence;
- lets a client contact any node, distinguishing leader from follower and redirecting writes to the leader;
- allows the cluster to grow without downtime through an explicit, operator-driven join protocol; and
- recovers correctly after crashes from durable, checksummed on-disk state.

The problem is intentionally bounded: single-key operations, one keyspace, no sharding, no multi-datacentre replication and no query language.

1.3 Project Objectives

The project has eight objectives, derived from the Software Design Document (SDD):

1. Implement a correct, understandable Raft core (leader election, log replication, commit, persistence) from scratch in Go.
2. Support configurable N-node clusters rather than a hard-coded cluster of three.
3. Support safe dynamic addition of nodes through an explicit join protocol with a non-voting learner stage and promotion.
4. Provide a strongly consistent PUT/GET/DELETE API over HTTP/JSON, backed by the replicated log, with leader redirection.
5. Provide durable storage (write-ahead log plus snapshots) with crash recovery.
6. Demonstrate professional software engineering: modular architecture, automated tests, CI, documentation and Architecture Decision Records (ADRs).
7. Demonstrate distributed-systems testing discipline: fault injection, partition simulation and race detection.
8. Produce a system that all four team members can fully explain, rather than one assembled from opaque libraries.

1.4 Scope

Every capability is classified with the MoSCoW scheme so that scope decisions are explicit and can be defended during evaluation. Table 1.1 summarises the classification; optional items must never delay the mandatory system.

Table 1.1: Scope classification (SDD section 5)

Tier	Capabilities
MUST	Leader election, terms and heartbeats; log replication, commit and applied indexes; persisted Raft metadata; configurable N-node clusters; PUT/GET/DELETE; WAL-based durability and crash recovery; snapshots and log compaction; dynamic join with learner stage and promotion; gRPC + Protobuf transport; HTTP API and CLI; structured logging; fault-injection framework; race-detector-clean code; Docker Compose deployment; CI; ADRs and diagrams.
SHOULD	Safe node removal; basic Prometheus-style metrics; read-index (linearizable) reads; shared-secret token on the join endpoint.
MAY	Chunked snapshot transfer; extra metrics (latency histograms); benchmark suite; watches, TTLs, compare-and-set, simple transactions.
Out of scope	etcd API compatibility, sharding, multi-datacentre replication, distributed SQL, production-grade authentication/TLS, automatic discovery, a dashboard as the primary deliverable.

1.5 Organisation of this Report

Chapter 2 states the requirements and the system model. Chapter 3 describes the development methodology and the design: architecture, the six UML views, the Raft protocol rules, persistence, dynamic membership, interfaces, deployment, safety invariants and failure handling. Chapter 4 reports implementation progress and project management: status, roadmap, effort and capacity, team organisation and risks. Chapter 5 presents the test strategy and the evaluation plan. Chapter 6 concludes and lists future work. The appendices give the HTTP API, the full phase breakdown, build instructions and a glossary.

2 Requirements and System Model

2.1 Stakeholders and Use Cases

Four groups have an interest in the system (Table 2.1). Two of them are simulated personas used to derive requirements: the client application and the cluster operator.

Table 2.1: Stakeholders and their interests

Stakeholder	Interest
Client application (simulated)	Issues PUT, GET and DELETE; expects strong consistency and clear leader-redirect behaviour.
Cluster operator (simulated)	Deploys clusters, joins nodes, observes cluster health through the CLI, logs and metrics.
Course instructor / evaluator	Correctness, engineering rigour, documentation quality and demonstrable understanding, not only a working demo.
Student engineers and future maintainers	Learn distributed systems and software-engineering practice; the codebase must be understandable without institutional knowledge, hence ADRs and documentation.

Figure 2.1 shows the use case view. A *client* writes, reads and deletes keys and checks node status; writes include the internal use case "Propose command", which in turn includes replicating the log to followers, reaching a majority and applying the entry to the state machine. A *cluster operator* views cluster status (leader, term, peers), monitors logs and manages nodes (start, stop, restart, and later join or remove). The remaining actor is the set of other Raft nodes, which interact through gRPC. Leader election, heartbeats and failure recovery are automatic behaviours of the system and are not triggered by users.

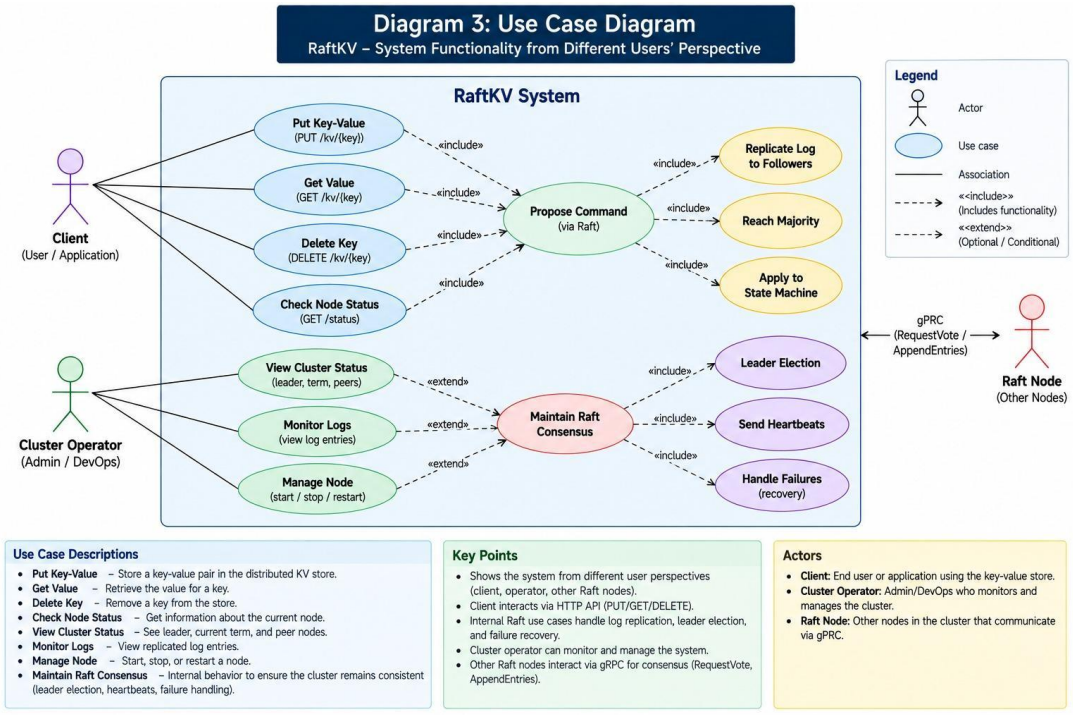


Figure 2.1: *Use case diagram: client, cluster operator and peer Raft nodes.*

2.2 Functional Requirements

Table 2.2 lists the functional requirements with their priority and the stage of the delivery plan in which each is realised, which gives direct requirements-to-plan traceability.

Table 2.2: Functional requirements and where they are planned

ID	Requirement	Priority	Planned in
FR-1	Elect exactly one leader per term among voting members, using randomized election timeouts and term-based voting.	MUST	Stage 3 (3.2)
FR-2	Replicate client writes as log entries with AppendEntries; commit an entry only after acknowledgement by a quorum of voters.	MUST	Stage 3 (3.3)
FR-3	Expose PUT, GET and DELETE over HTTP; writes and deletes become Raft log commands; GET is served by the leader.	MUST	Stages 2, 4 (4.2)
FR-4	Support N-node clusters; N is set at bootstrap time, not hard-coded.	MUST	Stages 1, 4 (4.4)
FR-5	Let a new node join a running cluster (`raftkv --join <addr>`) as a non-voting learner.	MUST	Stage 6 (6.2)
FR-6	Promote a learner to voter only once it has caught up to within a bounded distance of the leader's commit index.	MUST	Stage 6 (6.3)
FR-7	Remove a voting member safely (`raftctl cluster remove`), updating quorum requirements.	SHOULD	Stage 9 (9.2)
FR-8	Persist currentTerm, votedFor and log entries durably before acknowledging, so a restarted node recovers a state consistent with what it acknowledged.	MUST	Stage 5 (5.1, 5.2)
FR-9	Periodically snapshot the state machine and compact the log; transfer a snapshot to a lagging or new node.	MUST	Stage 5 (5.3)
FR-10	Redirect a write sent to a non-leader to the current leader; never drop it silently or forward it silently.	MUST	Stage 4 (4.3)
FR-11	Expose cluster status, current leader and member list through the HTTP API and the CLI.	MUST	Stage 4 (4.2, 4.3)
FR-12	Never commit a write when fewer than a quorum of voting members are reachable.	MUST	Stages 3, 7 (3.3, 7.2)

2.3 Non-Functional Requirements

Table 2.3: Non-functional requirements and how each is verified

ID	Requirement	Verification
NFR-1	**Safety over availability.** Raft safety properties are never violated to preserve availability.	Invariant-driven property tests (S1–S7) and failure tests.
NFR-2	**Testability.** Every module is testable in isolation and inside a running cluster, including under injected faults.	Interface-based design; in-process cluster harness.
NFR-3	**Understandability.** All four members understand the code and design; nothing is copy-pasted from a reference implementation.	Cross-review of every module; ADRs.
NFR-4	**Concurrency correctness.** All concurrent code passes the Go race detector consistently.	`go test -race ./...` is a mandatory CI gate.
NFR-5	**Observability.** Role, term, commit index and membership can be inferred from logs and metrics without a debugger.	Structured JSON logs; `/metrics` endpoint.
NFR-6	**Modularity.** The Raft core imports no HTTP or CLI packages; storage imports no gRPC.	Import-boundary check in CI.
NFR-7	**Bounded scope.** Optional features never delay the mandatory system.	MoSCoW classification; pinned out-of-scope list.
NFR-8	**Reasonable performance.** A sanity bound of single-digit to tens of milliseconds local commit latency on a 3–5 node Docker cluster; no throughput SLA is claimed.	Basic latency measurement (Chapter 5).
NFR-9	**Documentation.** A newcomer can deploy a cluster and join a node using only the documentation.	Documentation walkthrough review.

2.4 System Model and Assumptions

A distributed-systems design is only meaningful relative to a stated system model. Table 2.4 lists the assumptions on which the design depends; each is also a documented limitation.

Table 2.4: Assumptions of the system model

ID	Assumption	Consequence
A1	Crash-recovery, not Byzantine: nodes may crash and restart but do not behave maliciously.	No Byzantine fault tolerance is attempted.
A2	Asynchronous network: messages may be delayed, dropped or reordered.	Safety holds under arbitrary delay and loss; liveness assumes the network is not partitioned forever.
A3	Local, bounded clock drift; timeouts use local timers.	No clock synchronization is required; timeout ranges are chosen generously.
A4	Durable storage is trustworthy but not infallible.	Checksums detect corruption; corruption found at startup is a fatal error requiring operator action.
A5	Membership changes are processed one node at a time.	Concurrent multi-node reconfiguration (joint consensus) is out of scope.
A6	Clients are simple HTTP callers without request IDs.	Idempotency of retried writes is not guaranteed; documented as a known limitation.
A7	The network is trusted.	No authentication or TLS in the mandatory scope.
A8	Single keyspace, single-key operations.	No multi-key transactions.

3 Methodology and Design

3.1 Development Methodology

RaftKV is built as a disciplined, specification-driven engineering project rather than an exploratory prototype. The method has four elements.

- **Specification first.** The Software Design Document (SDD) defines requirements, architecture, protocols, data models, testing strategy and roadmap before implementation. Every ambiguity the design resolves is marked as an assumption and listed for team ratification; changes after ratification are recorded as Architecture Decision Records (ADRs) instead of silent edits.
- **Interface first, implementation second.** The three cross-cutting interfaces (state machine, storage, transport) are defined as signatures only and frozen after a synchronous team review. This lets four people work in parallel: the key-value engine can be built before Raft exists, and Raft can be tested against fake storage and an in-process transport.
- **Staged delivery with exit criteria.** Work is divided into eight stages, each with explicit exit criteria and hard gates. For example, Stage 3 (Raft core) may not start until the interface freeze has happened.
- **Failure paths are first-class.** A feature is not done merely because it compiles or passes a happy-path demo. Failure tests, race-detector runs and peer review are part of the Definition of Done.

Table 3.1 summarises the engineering practices and their current status.

Table 3.1: Engineering practices and their status

Practice	How it is applied	Artefact	Status
Branching and pull requests	`main` is always green; feature branches `area/short-description`; every merge by pull request with at least one reviewer (two for `internal/raft` and `internal/membership`).	GitHub PRs, branch protection	Partly in place
Continuous integration	Build, vet, test and race detector on every pull request; a Docker Compose smoke test is planned.	`.github/workflows`	In place
Invariant-aware review	Each PR states which safety invariants (S1–S7) it touches and ticks the Definition of Done.	PR template	Planned
Architecture Decision Records	One file per decision (context, decision, consequences).	`docs/adr/`	Partly in place
Issue tracking and roadmap	Stage = milestone, phase = epic, task = issue; weekly sprints; estimates and dependencies.	GitHub Projects board	In progress
Documentation	README, contracts, API reference, per-stage status documents, UML sources.	`docs/`	In place

3.2 System Architecture

3.2.1 Layered Responsibility Model

Each RaftKV node is a single process organised in the layers of Figure 3.1. A request enters the **API layer**, which parses and validates it and hands commands to the Raft core through a narrow surface (`Propose` and `Status`). The **Raft core** manages elections, replication, commitment and the application of committed entries. Committed entries are delivered to the **state machine**; term, vote and log entries are persisted through the **storage engine** before any response that depends on them is sent; and RPCs travel through the **transport layer**. The **membership manager** owns the cluster configuration and proposes configuration changes as ordinary log entries, so that they pass through the same commit machinery as client writes.

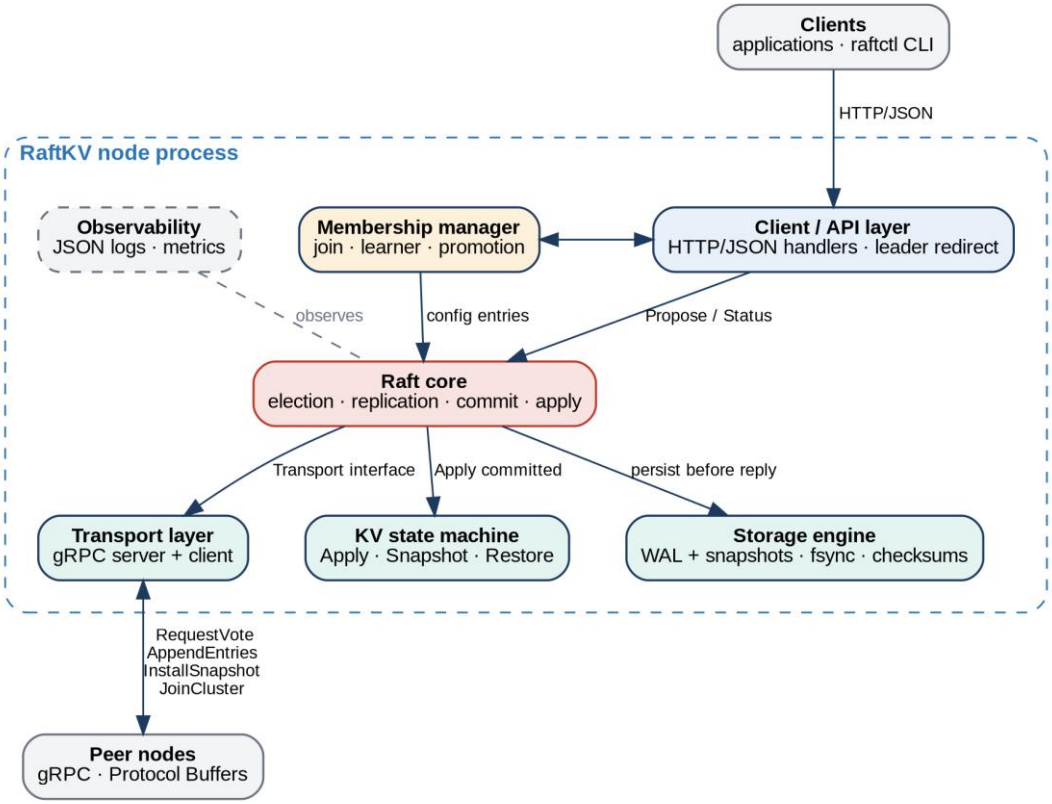


Figure 3.1: *Layered architecture of a single RaftKV node.*

3.2.2 Technology Stack

Table 3.2: Technology choices and rationale

Concern	Choice	Rationale
Language	Go	Goroutines and channels suit a concurrent protocol; the built-in race detector is a mandatory quality gate.
Node-to-node RPC	gRPC with Protocol Buffers	Strongly typed contracts, built-in streaming for later snapshot transfer, mature Go tooling.
Client interface	HTTP/JSON REST (`/kv/{key}`, `/cluster/*`)	Simple for clients and for the thin `raftctl` CLI.
State machine	In-memory map behind the `StateMachine` interface	Deterministic, trivially snapshot-able; keeps consensus independent of key-value semantics.
Persistence	Single append-only write-ahead log plus snapshot files per node, with checksums	Simple to reason about; compaction already requires periodic rewriting, so segmented logs add no benefit at this scale.
Runtime	Docker Compose, one container and one named volume per node	Realistic crash and restart behaviour for integration tests and demos.
Quality gates	GitHub Actions: build, vet, test, `-race`, lint	Every pull request is validated automatically.

3.2.3 Modules and Coupling Rules

The code is divided into packages with one responsibility each. The dependency rules below are enforced by code review and by an automated import-boundary check, not only by convention (Figure 3.2).

Table 3.3: Module responsibilities

Module	Responsibility	Depends on
`internal/raft`	Leader election, term management, log replication, commit/applied index tracking, persistent Raft state.	`internal/storage`; defines the `Transport` and `StateMachine` interfaces it needs.
`internal/storage`	Write-ahead log, snapshot read/write, crash recovery, log compaction.	Filesystem only; stores opaque bytes and does not interpret key-value semantics.
`internal/state`	Deterministic key-value engine; produces and restores snapshots.	Nothing: no knowledge of Raft, transport or networking.
`internal/transport`	gRPC server and client; conversion between Protobuf messages and Go structures.	Generated protocol code; implements the interface defined by `internal/raft`.
`internal/membership`	Node identity, cluster configuration, join protocol, learner catch-up tracking and promotion.	Raft core, transport, storage.
`internal/api`, `cmd/raftctl`	HTTP handlers, leader redirection; thin command-line wrapper.	Raft core (`Propose`, `Status`), membership manager.
`internal/fault`, `tests/`	Fault-injection hooks and cluster test harness.	Test-only; wraps the transport and controls node processes.

Module	Responsibility	Depends on
`internal/observability`	Structured JSON logging and a small metrics surface.	Consumed by all modules through a small interface, not a global singleton.

- `internal/raft` may depend on `internal/storage`, but **not** directly on transport, API or membership: it reaches the network through an interface that it defines itself (dependency inversion).
- `internal/state` has no knowledge of Raft or networking; it only implements the `Apply`, `Snapshot` and `Restore` contract.
- Only `internal/api` and `internal/membership` may depend on both the Raft core and the transport. A small composition root (`internal/server`) wires the modules into a running process.

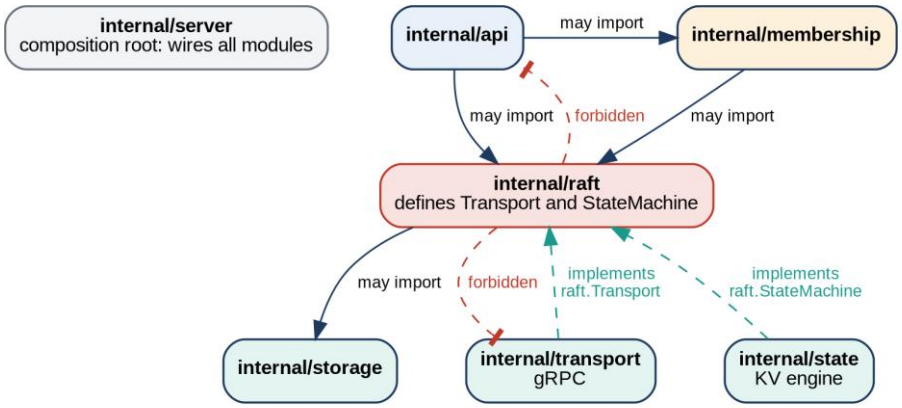


Figure 3.2: *Module dependency rules: allowed imports, interface implementation and forbidden dependencies.*

3.3 Structural Design

Figure 3.3 shows the class view from the first design baseline: `RaftNode` holds the Raft state (term, vote, log, commit and applied indexes, and per-peer `nextIndex`/`matchIndex`) and depends on three interfaces, `StateMachine`, `Storage` and `Transport`, which are implemented by `KVStore`, `FileStorage` and `GRPCTransport` respectively. The `HTTPServer` uses only `RaftNode`. `LogEntry` and `Command` are plain data classes, and the four Raft RPC messages carry exactly the fields of the Raft paper.

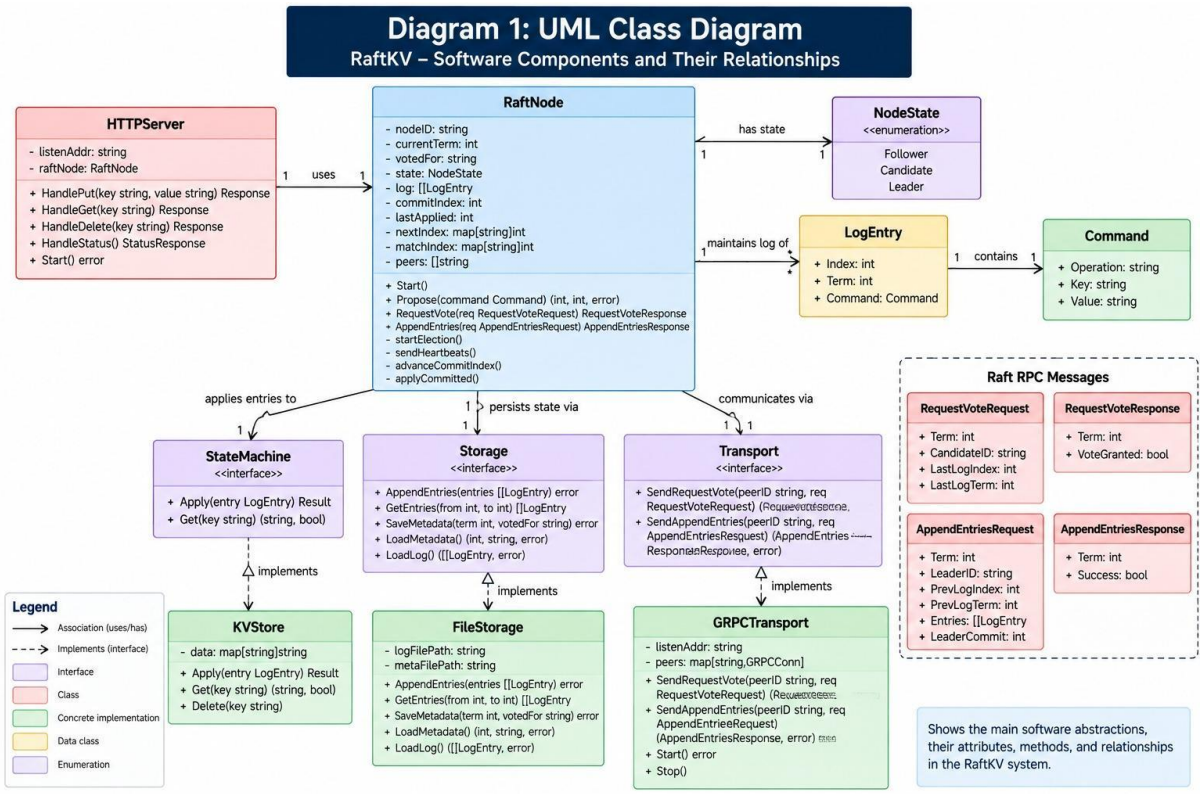


Figure 3.3: *UML class diagram of the RaftKV software abstractions (design baseline).*

The class diagram shows the subset needed for the first design baseline. The frozen interface contracts of the SDD extend it in four ways, summarised in the following table: the state machine gains `Snapshot` and `Restore`; storage gains `TruncateFrom` (for conflict resolution) and snapshot persistence and compaction; the transport gains `InstallSnapshot`; and a `MembershipManager` is introduced.

Table 3.4: Core interfaces in the frozen contracts

Interface	Implemented by	Operations
`StateMachine`	Key-value engine (`internal/state`)	`Apply(entry)` returns a result; `Snapshot()` returns bytes; `Restore(bytes)`.
`Storage`	File-backed WAL (`internal/storage`)	`SaveState`, `LoadState`, `AppendEntries`, `TruncateFrom`, `GetEntries`, `LastIndex`, `SaveSnapshot`, `LoadLatestSnapshot`, `CompactBefore`.
`Transport`	gRPC client (`internal/transport`)	`SendRequestVote`, `SendAppendEntries`, `SendInstallSnapshot`.
`Raft`	`RaftNode` (`internal/raft`)	`Propose(cmd)` returns index, term, isLeader; `Status()`; `Step(msg)` as inbound RPC entry point; `Stop()`.
`MembershipManager`	`internal/membership`	`HandleJoinRequest`, `PromoteIfCaughtUp`, `RemoveMember` (SHOULD), `CurrentConfig`.

3.4 Behavioural Design

3.4.1 Node Roles and State Transitions

Every node starts as a **follower**. If the election timeout expires without valid leader activity, the node becomes a **candidate**, increments its term, votes for itself and requests votes from its peers. A candidate that receives votes from a majority of voters becomes the **leader**. A node that receives a valid message from a current leader, or observes a higher term in any request or response, steps down to follower (Figure 3.4). Terms increase monotonically and the highest observed term is adopted by every node. Election timeouts are randomized in the range of 150–300 ms to make repeated split votes unlikely, and the leader sends heartbeats (empty AppendEntries) at roughly one tenth of the minimum election timeout.

A *learner* is deliberately **not** a fourth Raft state. It is a membership attribute layered on top of the follower role: a learner runs the same replication logic as a follower, but the membership manager excludes it from vote eligibility and from quorum arithmetic. This keeps the core state machine textbook-standard and is recorded as ADR-0001, pending team confirmation.

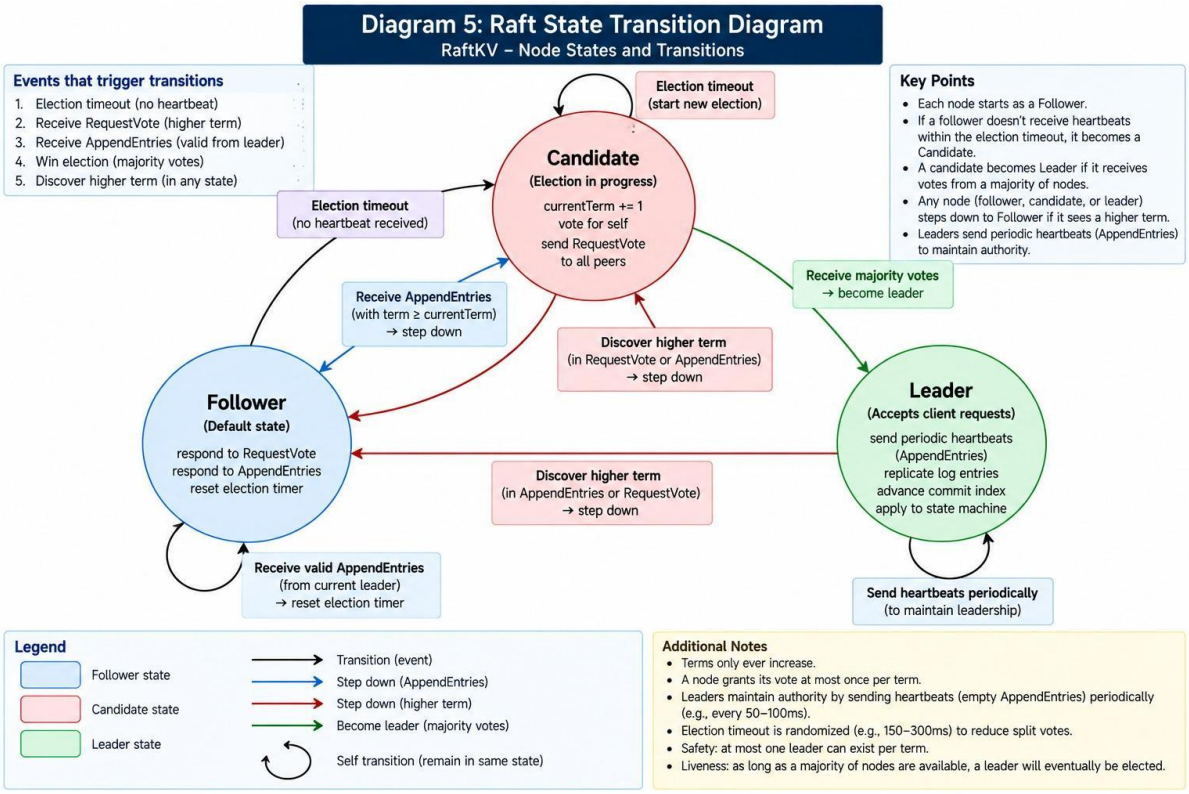


Figure 3.4: *Raft node state transitions among follower, candidate and leader.*

3.4.2 Leader Election

1. A follower that receives no valid AppendEntries or vote request within its randomized timeout becomes a candidate for a new term.
2. The candidate increments `currentTerm`, votes for itself (persisting the vote) and sends RequestVote to all voting members.
3. A node grants **at most one vote per term** and persists `votedFor` before replying. It grants the vote only if the candidate's log is at least as up to date as its own, comparing the term of the last entry first and the index second. This election restriction guarantees that a leader holds every committed entry.

4. A candidate that gathers votes from a majority of voters becomes leader, initialises `nextIndex` and `matchIndex` for each follower, sends an immediate heartbeat and appends a no-op entry in its own term so that entries from earlier terms can be committed.
5. If votes split, the randomized timeouts make the nodes retry at different moments with a higher term until one candidate wins.
6. Any request or response carrying a higher term causes the receiver to adopt that term and step down.

3.4.3 Log Replication and Commit

The write path (Figures 3.5 and 3.6) proceeds as follows. The client sends PUT to the HTTP API of the leader, which calls `Propose`. The leader appends the command to its own log (persisting it) and sends AppendEntries to every follower. Each follower checks that its log contains an entry at `prevLogIndex` with term `prevLogTerm`; if so it truncates any conflicting suffix, appends the new entries, persists them and acknowledges. If the check fails, the leader decrements that follower's `nextIndex` and retries until the logs match, which repairs divergent followers after a leader change.

The leader advances `commitIndex` when a majority of voters (itself included) store an entry **and** the entry belongs to the leader's current term; entries from earlier terms are committed only indirectly, as a consequence of committing a current-term entry. Committed entries are applied to the state machine strictly in order, and only then does the leader answer the client. Followers learn the new commit index from the next AppendEntries or heartbeat and apply the entry to their own state machines. In a three-node cluster a majority is two nodes.

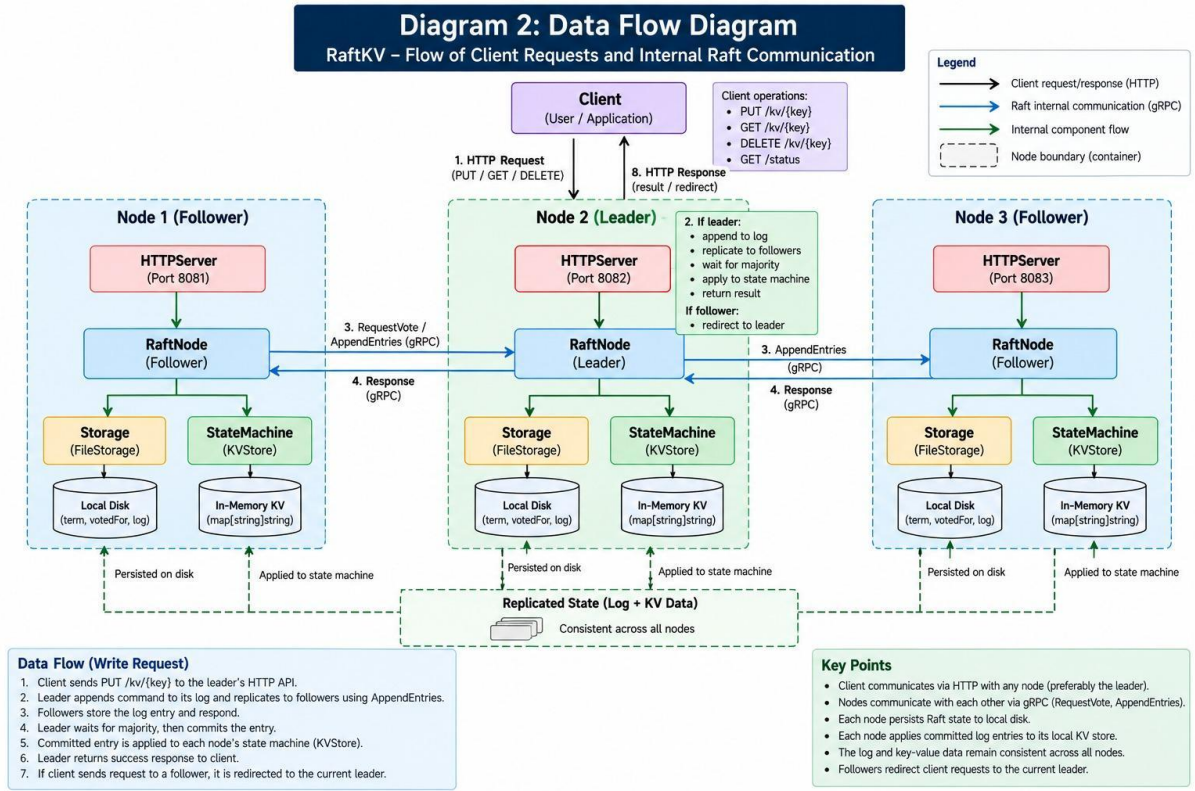


Figure 3.5: *Data flow of a client operation through the leader, follower logs, persistence and state machine.*

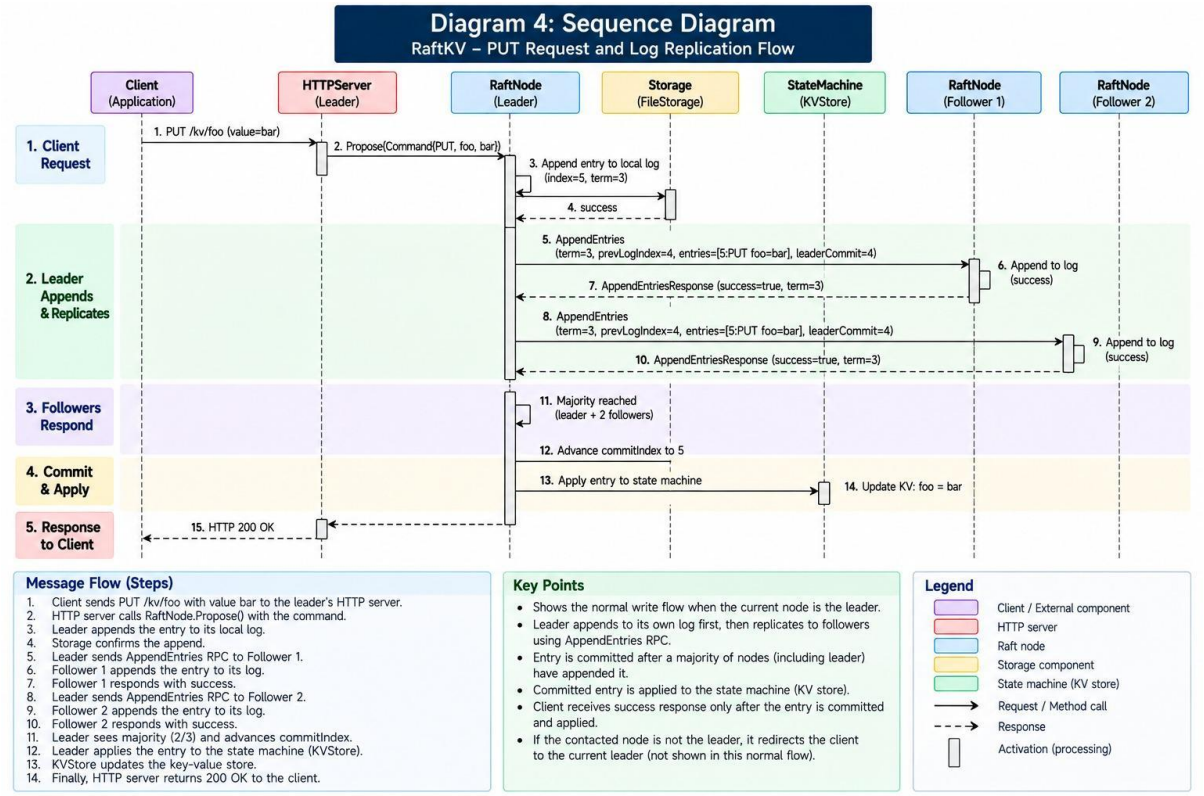


Figure 3.6: *Sequence of a PUT request: replication, majority commit and state-machine application.*

3.4.4 Client Requests and Leader Redirection

A client may contact any node. Writes (PUT and DELETE) are accepted only by the leader. A follower answers with a structured "not leader" error that carries a leader hint, and `raftctl` follows that hint once automatically. The design deliberately **redirects** instead of forwarding the request internally: forwarding would hide which node did the work, blur the attribution of timeouts ("did the write fail, or did the forward fail?") and add a failure mode when the leader changes mid-forward. The following table lists how the API reports each situation; no error path leaves ambiguity about whether a write was committed.

Table 3.5: Client-facing responses and their effect on the log

Situation	Response	Effect on the Raft log
Write committed and applied	200 OK	Entry committed on a quorum and applied.
Write sent to a follower	409 Conflict with `{"error": "not leader", "leaderHint": …}` (307 redirect is the documented alternative)	None.
Quorum unavailable	503 Service Unavailable, "quorum unavailable"	None; the write is never acknowledged and later dropped.
Malformed request (bad JSON, missing key)	400 Bad Request	None; the Raft log is not touched.
Join validation failure	409 Conflict with a specific reason	None; no partial application.

3.4.5 Read Consistency

In the mandatory scope, GET is answered directly from the believed leader's applied in-memory state without going through the log. This is simple and fast, but it has a known gap: a leader that has just lost leadership (for example during a partition) and has not yet seen a higher term could serve a stale read for a bounded window. Properties such as "no committed value disappears" concern what is *written and committed*; they do not by themselves guarantee that every read returns the latest value (the distinction between consistent writes and linearizable reads is discussed in [4]). The gap is documented explicitly, and a read-index or leader-lease mechanism that closes it is classified as a SHOULD-level stretch goal.

3.5 Persistence and Recovery

Raft's safety depends on durable state: a node must never forget a vote it has granted or an entry it has acknowledged. RaftKV therefore follows one rule: **no vote response or AppendEntries acknowledgement is sent before the data it depends on has been written and fsync'd**. State is stored in a single append-only write-ahead log (WAL) per node. Each record has the frame shown in the following table.

Table 3.6: Write-ahead log record format

Field	Purpose
`length`	Frame length, used to detect a partially written record.
`checksum`	CRC over the payload; detects torn or corrupted writes.
`recordType`	`STATE` (currentTerm and votedFor) or `LOG_ENTRY`.
`payload`	Serialized Raft metadata, or a log entry (index, term, command type, command).

On startup the node reads the WAL sequentially and stops at the first checksum failure or truncated frame; everything before that point is the recovered log. This correctly handles the classic failure of a crash in the middle of writing the last record. Corruption found anywhere else, an inconsistency between a snapshot and the WAL, or a disk-full condition is treated as a fatal error: the node logs loudly and refuses to serve rather than pretend to have recovered. Recovery only reads persisted state until it has been validated, so a second crash during recovery is harmless and the next restart simply repeats it.

Snapshots and compaction. Without compaction the WAL would grow without bound and a new node would have to replay the entire history. Every N committed entries (configurable, so that behaviour is deterministic and testable) the node serialises the state machine together with `lastIncludedIndex` and `lastIncludedTerm`, writes a checksummed snapshot file and updates a small pointer to the latest snapshot. Log entries up to `lastIncludedIndex` may then be discarded. The invariant is that compaction must never run ahead of what has been applied locally. A lagging or new node is brought up to date with an `InstallSnapshot` RPC that carries the whole snapshot in one message, followed by the remaining log tail. Sending the snapshot as a single message is a known scalability limit; chunked transfer is a MAY-level optimisation.

3.6 Dynamic Membership

Membership changes are represented as special entries in the **same replicated log** used for key-value commands (`CONFIG_ADD_LEARNER`, `CONFIG_PROMOTE_VOTER`, and `CONFIG_REMOVE` for the optional removal). A configuration change takes effect only once it is committed, so it inherits all the safety properties of ordinary entries and reuses the existing replication machinery instead of introducing a second consensus path. The design follows the single-server change approach of the Raft dissertation [2]: **only one membership change may be in flight at a time**, and a second request while one is pending is rejected. Joint consensus, which allows arbitrary simultaneous changes, is rejected as unnecessary complexity for this project. The cluster configuration (voters, learners and addresses) is itself durable state, so a full-cluster restart recovers the correct membership and not merely each node's individual log.

The join protocol is shown in Figure 3.7. A new node contacts any existing member; a follower answers with the leader hint and the node retries against the leader. The leader validates the request (no change in progress, node ID not already in use, request reached the leader), then appends and commits an `ADD_LEARNER` entry. It starts replicating to the learner with AppendEntries, or with InstallSnapshot if the learner's log is empty or far behind. The leader tracks the learner's `matchIndex`; once it is within a small configurable distance of the commit index, the leader appends a `PROMOTE_VOTER` entry. The promotion entry is committed under the **old** quorum, so the new node does not vote on its own promotion. Only after the entry is committed and applied does the node count towards the quorum, which is recomputed as $\lfloor N/2 \rfloor + 1$.

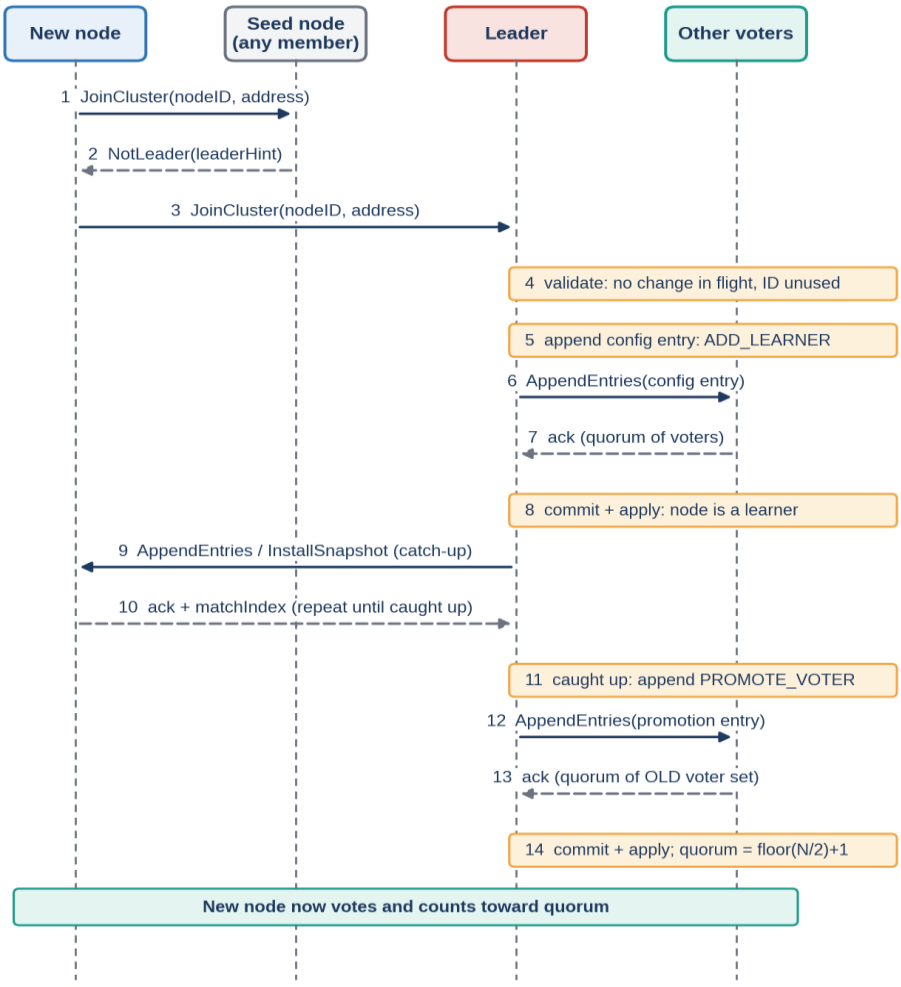


Figure 3.7: *Dynamic join protocol: learner admission, catch-up and promotion to voter.*

Two properties make this safe. First, a learner never votes and never counts for quorum, so a node that is still catching up cannot influence elections or commits. Second, because the learner receives replication before it is promoted, adding it never reduces availability: the quorum size changes only at the instant the promotion commits. Removal of a voter follows the same pattern in reverse. Removing the current leader is rejected in the first implementation (the operator must first make another node the leader), because a leader that removes itself could leave the cluster leaderless mid-transition.

3.7 Interfaces and Data Structures

The persisted data structures are listed in the following table. Log entries carry an opaque command so that the storage layer never interprets key-value semantics.

Table 3.7: Persisted data structures

Structure	Fields	Notes
Raft metadata	`currentTerm` (unsigned 64-bit, monotonic); `votedFor` (node ID or none)	Persisted before any reply that depends on it.
Log entry	`index`, `term`, `commandType` (`KV_PUT`, `KV_DELETE`, `CONFIG_ADD_LEARNER`, `CONFIG_PROMOTE_VOTER`, `CONFIG_REMOVE`, `NOOP`), `command` (bytes)	Applying an already-applied index is a no-op, so retries are safe.
Membership	`nodeID`, `address`, `role` (`VOTER`, `LEARNER`), `status`, `lastKnownProgress`	Durable through config entries and snapshots.
Snapshot	`lastIncludedIndex`, `lastIncludedTerm`, `serializedState`, `checksum`	Allows the WAL to be compacted up to `lastIncludedIndex`.
Key-value record	`key` (string), `value` (bytes), optional `revision`	`revision` is reserved for compare-and-set (MAY) and unused in the mandatory scope.

Nodes communicate through two gRPC services, listed in the following table. The message fields mirror the Raft paper: RequestVote carries the candidate's term, identifier and last-log information; AppendEntries carries the leader's term and identifier, the previous-log index and term, the entries and the leader's commit index. The client-facing HTTP endpoints are listed in Appendix A.

Table 3.8: Node-to-node RPCs

Service	RPC	Purpose
`RaftTransport`	`RequestVote`	A candidate asks for a vote in a new term.
`RaftTransport`	`AppendEntries`	Log replication and heartbeats.
`RaftTransport`	`InstallSnapshot`	Bring a lagging or new node up to date.
`MembershipTransport`	`JoinCluster`	A new node requests admission as a learner.
`MembershipTransport`	`RemoveMember` (SHOULD)	Remove a member from the cluster.

3.8 Deployment Design

Each node runs as an independent Docker container with its own named volume, so that stopping and restarting a container realistically simulates a node crash and recovery while the data survives (Figure 3.8). By default Docker Compose starts three processes; Compose profiles add a fourth and fifth node, which then join the running cluster as learners. Node n listens for clients on HTTP port 800n and for peers on gRPC port 900n. Nodes reach one another over a private Docker network in a full mesh. Clients may contact any node; no load balancer is required because non-leaders redirect writes.

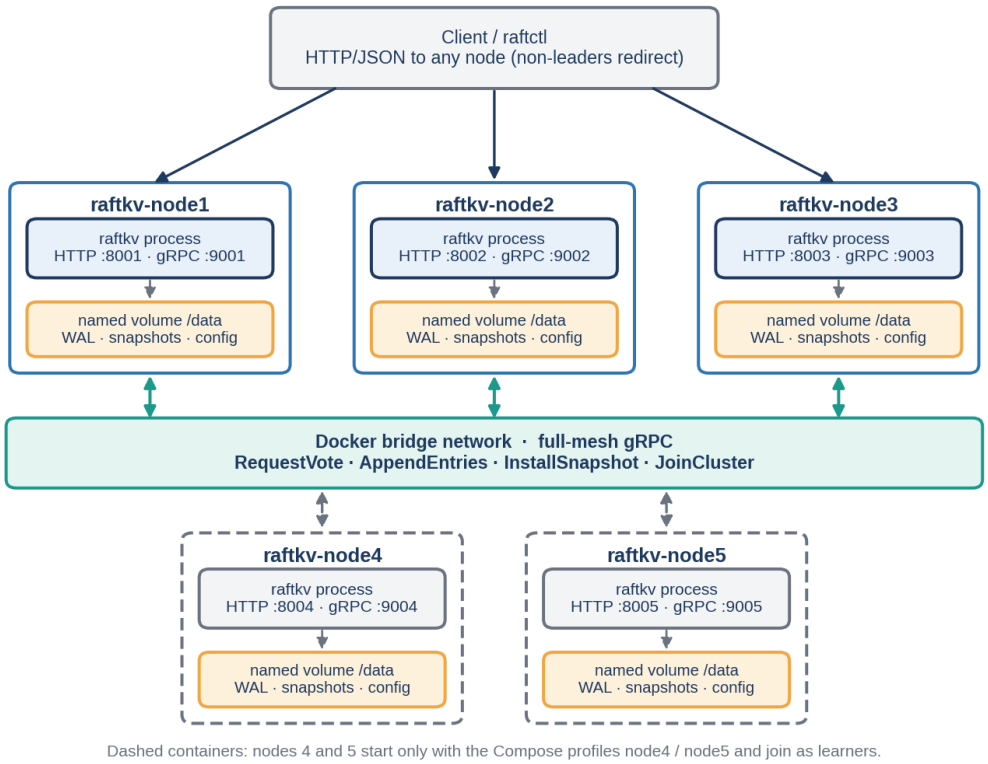


Figure 3.8: *Target deployment: one container and one volume per node on a Docker network.*

Each node is configured by flags that override environment variables, as listed in the following table. Starting a process does not by itself make it part of a Raft cluster; that is the role of the join protocol.

Table 3.9: Node configuration

Flag	Environment variable	Default	Meaning
`--node-id`	`RAFTKV_NODE_ID`	`node1`	Stable, unique node identifier.
`--peers`	`RAFTKV_PEERS`	empty	Peer list as `id=grpc-host:port`, comma-separated (bootstrap convention awaiting SDD confirmation).
`--data-dir`	`RAFTKV_DATA_DIR`	`./data`	Directory for durable state.
`--http-addr`	`RAFTKV_HTTP_ADDR`	`:8001`	Client API listen address.
`--grpc-addr`	`RAFTKV_GRPC_ADDR`	`:9001`	Node-to-node listen address.

3.9 Correctness Properties

The following table lists the properties the implementation must never violate. Every Raft-related design decision is checked against it, every pull request that touches consensus states which of them it affects, and the property tests of Chapter 5 check them across randomized fault runs.

Table 3.10: Raft safety invariants

ID	Invariant	Where enforced
S1	**Election safety.** At most one leader can be elected in a given term.	Persisted vote-once-per-term; majority quorum.
S2	**Leader append-only.** A leader never overwrites or deletes entries in its own log.	Raft core log-write path.
S3	**Log matching.** If two logs contain an entry with the same index and term, all preceding entries are identical.	AppendEntries consistency check and conflict truncation.
S4	**Leader completeness.** A committed entry is present in the logs of all leaders of higher terms.	Election restriction on log up-to-dateness.
S5	**State machine safety.** No two servers apply different entries at the same index.	Follows from S1–S4 plus deterministic `Apply`.
S6	**Membership-change safety.** A configuration change takes effect only once committed; a learner never counts for quorum; at most one change is in flight.	Membership manager; commit-then-apply.
S7	**No commit without quorum.** An entry is committed only after acknowledgement by a majority of the *current* voting members.	Commit-index advancement in the Raft core.

These invariants describe Raft-level safety. They do not by themselves provide linearizable reads; that is the read-consistency caveat of Section 3.4.5.

3.10 Failure Handling

The required failure scenarios are mapped to expected behaviour in the following table. Each scenario is also an automated test in the plan (Chapter 5).

Table 3.11: Failure scenarios and expected behaviour

Scenario	Expected behaviour
Leader crash	Followers stop receiving heartbeats; an election timeout fires and a new leader is elected once a quorum votes. Uncommitted entries on the old leader that never reached a quorum are lost, which is correct because they were never committed.
Follower crash	No impact on availability while a quorum remains. The restarted follower catches up through AppendEntries, or through a snapshot if it is far behind.
Loss of quorum	Writes stop committing by design and the API returns 503. Reads may still be served from applied state with the consistency caveat. Once a quorum is restored, the cluster resumes.
Network partition	The minority side cannot elect a leader or commit. The majority side continues. When the partition heals, minority nodes discover the higher term, step down and resynchronise.
Node restart	The node reloads term, vote, log and any snapshot, rejoins as a follower and catches up.
Join during client writes	The join protocol runs concurrently with writes; the learner catches up by receiving the same AppendEntries as voters.
Crash while a learner catches up	On restart the learner resumes from its persisted state; the leader continues from the learner's `matchIndex`. No special resume protocol is needed beyond normal replication retry.
Crash during recovery	Recovery reads persisted, checksummed state and does not mutate it until validated, so it is safely repeatable.
Crash during a membership change	The configuration entry is either committed (and recovered like any entry) or was never committed and is absent; no partial membership state is possible.

3.11 Design Evolution and Key Decisions

The six diagrams were first produced for a fixed three-node baseline. While the SDD was written, the scope was aligned with the project's defining requirement, dynamic membership, and several details were tightened. The following table records the differences so that readers comparing the diagrams with this chapter are not misled.

Table 3.12: Evolution from the design baseline to the SDD

Aspect	Design baseline (diagrams)	Current design (SDD and repository)
Cluster size	Fixed at three nodes.	Configurable N nodes set at bootstrap.
Membership	Future work.	Mandatory: join, learner stage, promotion; removal as SHOULD.
Persistence	File-backed JSON log and metadata.	Checksummed, fsync'd write-ahead log plus snapshots.
Snapshots	Future work.	Mandatory, with InstallSnapshot for lagging and new nodes.
Ports	HTTP 8081–8083; gRPC 9001–9003 (diagram: 9091–9093).	HTTP 800*n* and gRPC 900*n* for node *n* (repository).

Aspect	Design baseline (diagrams)	Current design (SDD and repository)
Entry point	Load balancer in front of the nodes.	Clients contact any node; non-leaders redirect. No load balancer.
Follower write	Retry against the leader address.	Structured 409 with leader hint (307 documented as an alternative).

The principal design decisions, with the alternatives that were rejected, are summarised in the following table. Each is, or will be, recorded as an ADR.

Table 3.13: Key design decisions

Decision	Alternative considered	Rationale
Learner is a membership attribute, not a fourth Raft state	Add a Learner role to the state machine	Keeps the core protocol textbook-standard and easier to test; the join flow is still satisfied. Pending team confirmation (ADR-0001).
Redirect writes from followers	Forward internally to the leader	Simpler to implement and reason about; avoids a failure mode when leadership changes mid-forward.
Configuration changes as log entries, one at a time	Joint consensus	Reuses commit machinery; joint consensus is unnecessary complexity at this scale.
Single append-only WAL file per node	Segmented or per-entry files	Compaction already rewrites the file; no benefit from segmentation at this data volume.
Snapshot sent as one message	Chunked streaming	Acceptable limitation for small state; chunking is MAY-level.
Reads served by the leader's applied state	Read-index or leader lease from the start	Simple MVP; the stale-read window is documented and the stronger mechanism is a stretch goal.
gRPC for node-to-node traffic	HTTP/JSON between nodes	Typed contracts, streaming for later snapshots, mature tooling.
No authentication or TLS in the mandatory scope	Token or mTLS from the start	Trusted-network assumption (A7); recorded as a known limitation. A static join token is a cheap SHOULD-level hardening.
Crash loudly on storage errors	Degraded read-only mode	Silent degraded modes are harder to reason about and test in an educational project.

4 Implementation Progress and Project Management

4.1 Current Implementation Status

At the mid-semester point the Go repository contains the Stage 1 scaffolding and the Stage 2 standalone key-value engine (Figure 4.1). The state engine implements PUT, GET and DELETE, a deterministic `Apply`, `Snapshot` and `Restore`. The HTTP API and the gRPC services exist as skeletons: valid HTTP requests return 501 and every gRPC method returns *Unimplemented*, which is the intended behaviour until the consensus core exists. There is no Raft algorithm, write-ahead log, membership implementation or authentication yet, and Stage 3 has not started.

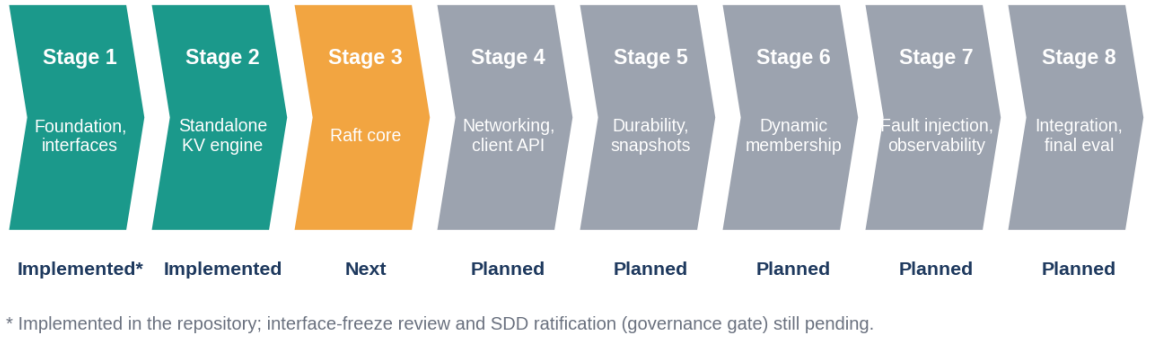


Figure 4.1: *Delivery status by stage at mid-semester.*

Table 4.1: Component status at mid-semester

Component	Status	Notes
Repository, build and tooling	Implemented	Go module; Makefile targets for build, vet, test, race, generate and lint; lint configuration; Dockerfile; MIT licence.
Continuous integration	Implemented	GitHub Actions workflow present in `.github/workflows`.
Protocol contracts	Skeleton	Protobuf schema and generated Go code committed; gRPC methods return *Unimplemented*.
HTTP API and `raftctl`	Skeleton	Routes and JSON schema defined; valid requests return 501; `raftctl` reports the server response.
Key-value state engine	Implemented	PUT, GET, DELETE; deterministic `Apply`; `Snapshot` and `Restore`.
Docker Compose	Skeleton	Three processes by default, a fourth and fifth through profiles; ports 800*n*/900*n*; one named volume per node. Processes are not yet joined into a Raft cluster.
Presentation demo console	Implemented (opt-in)	Local web console over the engine: PUT/GET/DELETE, key inspection, snapshot capture and restore. In-memory, disabled by default, no consensus.
Raft core (election, replication)	Not started	Stage 3.
Durable storage (WAL, snapshots)	Not started	Stage 5.

Component	Status	Notes
Dynamic membership	Not started	Stage 6.
Fault injection and observability	Not started	Stage 7.
Governance gates	Pending	Interface-freeze review, SDD ratification, ADR-0001 sign-off, branch protection.

Governance gate before Stage 3
The system-wide interface contracts are still a draft pending SDD review: Stage 2 was built against the existing state interface. The synchronous interface-freeze review and the SDD ratification are therefore the first two items of the next sprint, and Stage 3 may not start until they are done.

The repository layout follows the SDD. `cmd/` holds the two entry points (`raftkv`, `raftctl`); `internal/` holds one package per module of Table 3.3; `proto/` holds the Protobuf schema and generated code; `scripts/` holds helper scripts such as the node-adding script; `docs/` holds the implementation plan, contract decisions, the HTTP API reference and a status document per stage; `.github/workflows` holds CI. The build instructions are given in Appendix C.

4.2 Plan and Roadmap

The remaining work is organised into the stages of Table 4.2 and scheduled over ten weekly sprints from Monday 12 October to Friday 18 December 2026. In total the plan has 39 phases and 161 tasks: 55 in Stages 1–2, 98 in Stages 3–8 and 8 stretch tasks in Stage 9. Every task has an owner, a MoSCoW priority, an hour estimate, planned dates, dependencies and a reference to the SDD section it implements. Appendix B lists all phases.

Table 4.2: Stages, schedule and effort

Stage	Planned dates	Tasks	Hours	Exit criterion
1 Foundation, Interfaces & System Design	Built; close-out in Sprint 1	34	75	All interfaces frozen by a synchronous team review; SDD ratified; CI + branch protection active.
2 Standalone Key-Value Engine	Built; close-out in Sprint 1	21	47	Determinism + round-trip + import-boundary checks green in CI.
3 Raft Core (Election & Replication)	15 Oct – 06 Nov	21	116	No two leaders per term; committed entries never lost; commit only with quorum.
4 Networking & Client Integration	09 Nov – 27 Nov	15	66	PUT via raftctl on any node replicates and is readable cluster-wide.
5 Durability, Snapshots & Recovery	09 Nov – 04 Dec	16	84	Kill -9 and full-cluster restart lose no committed data.
6 Dynamic Membership (Join, Learner, Promotion)	23 Nov – 11 Dec	17	74	A 4th node joins a live cluster, catches up, is promoted, and survives restart.
7 Reliability: Fault Injection, Observability & Correctness	26 Oct – 11 Dec	16	81	Leader crash, follower crash, partition, restart scenarios pass; go test -race clean.
8 Integration, Documentation & Final Evaluation	07 Dec – 18 Dec	13	54	Demo reproducible from README; report and deck submitted.

Stage	Planned dates	Tasks	Hours	Exit criterion
9 Stretch Goals (post-MVP)	Unscheduled (post-MVP)	8	50	SHOULD/MAY items attempted only after the MVP is stable and documented (SDD sections 42-43).

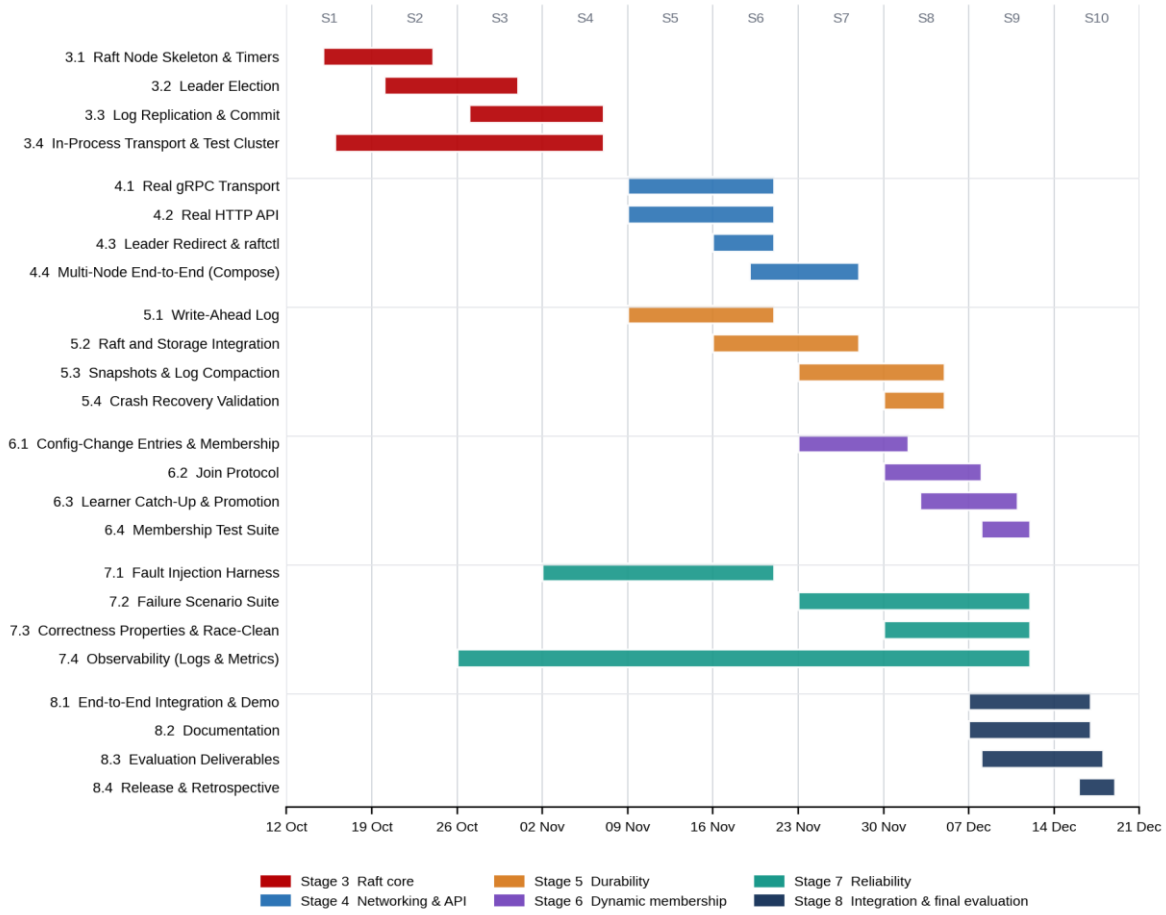


Figure 4.2: *Planned roadmap for Stages 3–8 at phase level (sprints S1–S10).*

4.3 Tracking with GitHub Projects

The plan is tracked on a GitHub Projects board linked to the repository, so that progress is visible to every team member and to the evaluator. The mapping is deliberately simple: a **stage is a milestone**, a **phase is an epic issue** with sub-issues, and a **task is an issue** whose identifier (for example `S3.2-01`: stage 3, phase 2, task 1) is traceable to an SDD section. Dependencies between tasks are recorded on each issue, and every item carries the custom fields of the following table.

Table 4.3: Project board fields and views

Element	Content
Status workflow	Backlog → Ready → In progress → In review → Done. *Ready* means all dependencies are done; *Done* means the pull request is merged, CI is green and the Definition of Done is met.
Custom fields	Role (owner), Priority (MUST/SHOULD/MAY), Area (module), Risk, Estimate (hours), Sprint, Start date, Target date, SDD reference, Depends on, Task ID, Phase.

Element	Content
Roadmap views	Stages and phases grouped by milestone; task-level timeline grouped by owner.
Working views	Kanban board with swimlanes per owner; current sprint; full backlog table; MVP scope split by priority; risks and gates; completed work with verification evidence; one view per team member.
Automation	Items enter the board as Backlog; closing an issue or merging a linked pull request moves it to Done.

The **Definition of Done** (SDD section 45) is applied to every feature: implementation merged through a reviewed pull request; unit tests; integration tests in a running multi-node cluster; failure tests where the feature has failure-relevant behaviour; documentation or ADR updated; logs and metrics present where relevant; `go test ./...` and `go test -race ./...` passing; and review by at least one other member (two for the Raft core and membership). A feature is not done merely because it compiles.

4.4 Effort, Capacity and Cut Line

Effort was estimated bottom-up per task. The remaining mandatory-and-recommended work in Stages 3–8 is about **475 hours** (432 h MUST and 43 h SHOULD); Stage 9 adds 50 h of stretch work that is attempted only after the MVP is stable. The SDD assumes four people at about 10 hours per week, which gives 400 hours of raw capacity over ten weeks and roughly 300 hours of effective build time after the 25 % typically consumed by coordination, review and debugging of distributed non-determinism (Figure 4.3).

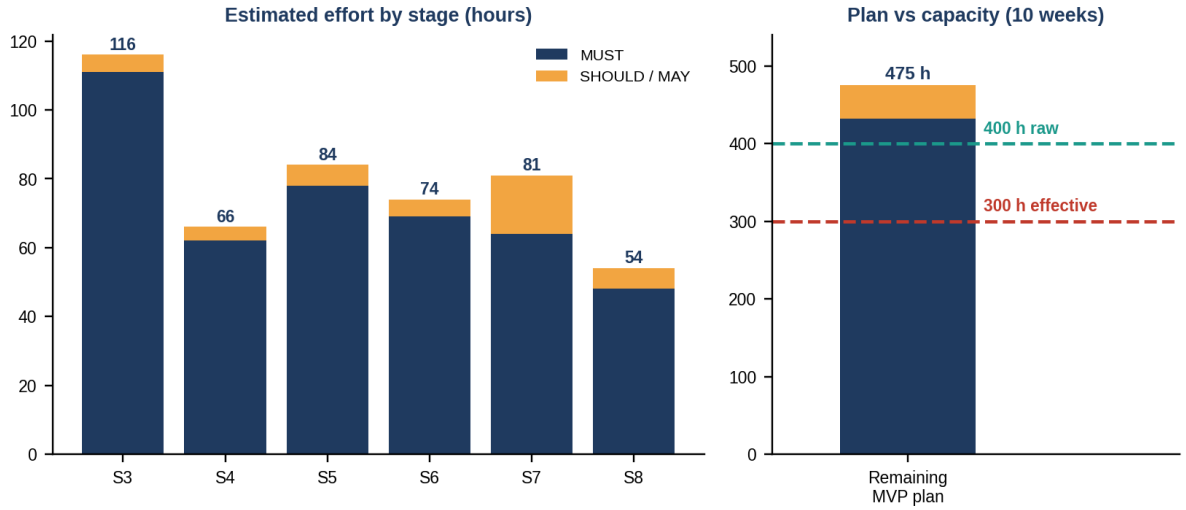


Figure 4.3: *Estimated effort by stage and comparison with team capacity.*

The plan therefore needs about 12 hours per person per week of build time, which is more than the capacity assumed in the SDD. This is the same gap that SDD section 47 predicted, and it is why the project status is reported as **at risk** rather than on track. Rather than hide the gap, the plan defines an explicit order in which scope is cut if the team falls behind (the following table). The items that are never cut are those that make the project a genuine distributed-systems exercise.

Table 4.4: Cut line if the schedule slips

Order	Cut	Saves	Rule
1	All SHOULD and MAY items, and all of Stage 9	43 h (+50 h stretch)	Never started before the MUST items of the same stage are done.
2	Second-wave fault scenarios (delayed or dropped packets; crash during recovery or during a membership change) and the nightly property job	about 12 h	Keep leader crash, follower crash, partition and restart.
3	Narrow the breadth of the property tests to invariants S1, S2, S4 and S7	about 4 h	The race-detector gate is never removed.
4	Agree a reduced membership demonstration scope with the instructor	negotiable	Needs instructor sign-off; it is not a silent cut.
Never	Election and replication correctness tests; WAL durability tests; `go test -race`; the join, learner and promotion happy path	—	SDD section 47.

4.5 Team Organisation

Ownership means *primary driver*, not an isolated silo: every mandatory feature is cross-reviewed by at least one other member, and all four members must be able to explain the whole system end to end. The following table shows the module ownership of the SDD and the hours the board assigns to each member for Stages 3–8 after balancing the load.

Table 4.5: Roles, modules and planned load (Stages 3–8)

Role	Primary modules	Cross-review with	Planned hours
Person 1 — Raft Core	`internal/raft`; state engine interface	Person 2 (storage integration), Person 4 (fault tests)	115
Person 2 — Storage and Membership	`internal/storage`, `internal/membership`	Person 1 (Raft integration), Person 4 (membership fault tests)	112
Person 3 — Networking and API	`internal/transport`, `internal/api`, `cmd/raftctl`, observability tasks	Person 1 (transport contract), Person 2 (join wiring)	98
Person 4 — Reliability and DevOps	`internal/fault`, `tests/`, CI, Docker, deployment	All (every module's tests are reviewed by its owner)	116
Whole team	Interface freeze, SDD ratification, race-clean build, report and presentation	—	34 (shared)

The weekly rhythm has four parts: shared planning at the start of each week (what lands, what depends on what); a mid-week integration session, because Raft, storage and transport are tightly coupled and "integrate at the end" is a known failure mode; rotating pull-request reviewers so that the same person does not always review the same author; and an end-of-week update of the documentation and ADRs together with a short scope check against the out-of-scope list.

4.6 Risk Management

Table 4.6: Risk register

#	Risk	Likelihood	Impact	Mitigation
1	Capacity gap: the plan needs about 12 h per person per week against 10 h assumed.	High	High	Cut line (Table 4.4); weekly burn check against the estimate field; protect Person 1 during Sprints 2–4.
2	Interfaces not yet frozen (S1.2-06) block Stage 3.	Medium	High	Freeze review and SDD ratification are the first two items of Sprint 1.
3	Log replication and conflict resolution take longer than planned.	High	High	Budgeted over two sprints; teammates own the test tasks so Person 1 stays on the critical path.
4	Membership (Stage 6) is underestimated.	High	High	Not started until Stages 3–5 are green; timeboxed; instructor checkpoint at the end of Sprint 7.
5	Fault harness built too late to validate earlier work.	Medium	High	In-process drop/delay hooks in Sprint 4; full harness from Sprints 4–6.
6	Tests under-invested relative to features.	Medium	High	Definition of Done requires failure tests; PR template enforces it.
7	Report and presentation time underestimated.	Medium	Medium	Stage 8 is explicit; ADRs are written incrementally with each pull request.
8	Scope creep towards etcd-like features.	Medium	Medium	Out-of-scope list pinned; Stage 9 clearly separated.

5 Testing and Evaluation

5.1 Test Strategy

Testing is organised in tiers, from fast unit tests to randomized fault-injection runs (the following table). Confidence in correctness comes from invariant-driven tests plus code review against the safety invariants of Section 3.9; exhaustive formal verification such as TLA+ model checking is out of scope for the available time.

Table 5.1: Test tiers

Tier	Examples	Environment
Unit	Vote-granting rules; log matching and conflict resolution; quorum size for various N; membership state transitions; WAL checksum and recovery; snapshot serialisation and restore.	Each module, by its owner, reviewed by all.
In-process integration	Three- and five-node election; replication without faults; PUT/GET/DELETE end to end through the API layer.	One test binary with in-process transport.
Docker integration	Bring up a multi-container cluster; join a real new container; kill the leader container and observe re-election through status polling.	Docker Compose.
Fault	Leader crash; follower crash; failures dropping below quorum; network partition; delayed or dropped packets; restart in the middle of catch-up.	Fault-injection harness.
Membership	Join; join during writes; learner catch-up; promotion; restart during catch-up; rejected or duplicate join.	In-process and Docker.
Property-style	Across many randomized fault runs: no two leaders in one term; committed entries never disappear or are overwritten; no write commits without quorum acknowledgement.	CI; an extended nightly run if it is too long for every pull request.

Two commands must pass in CI for every pull request: `go test ./...` and `go test -race ./...`. The race detector is non-negotiable for a Go concurrency project.

5.2 Fault-Injection Architecture

Fault injection is built into the architecture instead of being scattered through the Raft code as conditional branches. Because the Raft core talks to the network only through the `Transport` interface, a **test-only transport decorator** can drop, delay and reorder messages between specific node pairs and simulate partitions, slow links and message loss without manipulating the real network. **Process-level faults** (crash and restart) are driven at two tiers: with `docker stop` and `docker start` on a container for realism, and by stopping a Raft instance, discarding its in-memory state and reloading it from storage for fast in-process iteration.

Scenarios are data-driven (for example “at tick 5, kill the leader; at tick 8, restart it”), so that one harness can drive every scenario of Table 3.11. After each scenario the harness asserts convergence, absence of data loss, absence of split brain and correct quorum behaviour. The harness itself has tests, because a test framework that misreports faults is worse than none.

5.3 Verification at Mid-Semester

Because the consensus core is not implemented yet, no distributed-system test can pass or fail at this stage. What can be verified now are the Stage 1 and Stage 2 deliverables:

- the repository builds, vets and race-tests through the documented `make` targets, and a CI workflow is configured;
- the state engine meets its acceptance criteria: deterministic `Apply` (the same commands give the same state), a snapshot–restore round trip, isolation from Raft and network packages, and race-clean concurrent access;
- the API skeleton behaves as specified: valid requests return 501, gRPC methods return *Unimplemented*, and `raftctl` reports the server response;
- Docker Compose provisions three node processes with distinct ports and named volumes, and a fourth and fifth through profiles.

The repository keeps one status document per stage that records what was delivered and checked. Appendix C gives the commands that reproduce these checks.

5.4 Evaluation Plan and Metrics

No benchmark values have been measured, so none are claimed. The following table defines what will be measured once the corresponding stage is complete. Targets marked *proposed* are working hypotheses derived from the election timeout range and the NFR-8 sanity bound; they are not guarantees and will be replaced by measured values in the final report.

Table 5.2: Evaluation metrics and their status

Metric	Method	Target	Planned in	Status
Commit latency	Time from client PUT to HTTP 200 on a three-node Docker cluster (median and 95th percentile).	Single-digit to tens of ms (NFR-8 sanity bound)	Stage 8	Not yet measured
Failover time	Time from leader crash to the first successful write on the new leader.	A few election timeouts, below about 1 s (*proposed*)	Stage 7	Not yet measured
Recovery time	Time from node restart to caught up and serving.	Seconds for the small state of this project (*proposed*)	Stage 5	Not yet measured
Data loss	Committed writes missing after any injected crash.	Zero (invariants S4, S5)	Stages 5, 7	Not yet measured
Leaders per term	Number of leaders observed per term over randomized fault runs.	At most one (S1)	Stages 3, 7	Not yet measured
Availability under faults	Writes with a minority of nodes down; writes with quorum lost.	Succeed with a quorum; refused with 503 without one	Stage 7	Not yet measured
Join and promotion	Entries and time from `ADD_LEARNER` commit to promotion.	Within the configured threshold	Stage 6	Not yet measured

5.5 Analysis and Discussion

Consistency of the design. The six views describe one system from different angles and are mutually consistent. The use case view fixes the client and operator responsibilities; the class view defines the abstractions; the data-flow and sequence views describe the write path; the state-transition view describes the election lifecycle; and the deployment view fixes the physical topology. The differences between the first diagrams and the current design are listed in Table 3.12 so that no reader is misled.

Strengths. Consensus, state-machine semantics, persistence and transport are separate abstractions, which makes each testable alone and lets four people work in parallel. Every requirement is traceable to a stage, and every task to an SDD section. The design states its assumptions and limitations explicitly instead of overclaiming.

Known limitations. The mandatory scope has deliberate gaps, all documented: reads from the leader's applied state can be briefly stale after a leadership change; retried writes are not deduplicated (assumption A6); snapshots are sent as one message; there is no authentication or TLS; and operations are limited to single keys. Features such as sharding, transactions, multi-Raft and Kubernetes are explicitly outside the project.

Threats to validity. Evaluation will run on one host with a Docker network, so latencies will not represent a real wide-area deployment. Randomized fault tests are probabilistic and can miss rare interleavings; they complement, but do not replace, review against the invariants. Finally, the schedule depends on team capacity, which is the largest project risk (Section 4.6).

6 Conclusion

6.1 Summary of Work

This report presented the requirements, architecture, protocol design, delivery plan and test strategy of RaftKV, a dynamic-membership key-value store built on Raft. The design separates consensus, state machine, storage, transport, API and membership behind interfaces, specifies the election, replication, commit, persistence, snapshot and join/promotion rules, and records the safety invariants against which every change is reviewed. The implementation has reached the end of Stage 2: the tooling, protocol and API skeletons, Docker Compose and CI exist, and the standalone key-value engine implements deterministic apply, snapshot and restore.

6.2 Key Findings

- **Separation of concerns pays off early.** Because the state engine depends on nothing else, it could be implemented and verified before Raft exists, and Raft can be tested against fake storage and an in-process transport.
- **Dynamic membership can reuse ordinary consensus.** By representing configuration changes as log entries applied one at a time, and by treating a learner as a membership attribute rather than a new Raft state, the join protocol reuses the commit machinery and stays small enough to test thoroughly.
- **Durability has a precise contract.** No vote or acknowledgement is sent before the data it depends on is fsync'd; a checksummed WAL turns a torn final write into an expected, recoverable condition.
- **The plan is capacity-limited, and this was quantified early.** The remaining work needs about 12 hours per person per week against 10 assumed, so the project carries an explicit cut line and an at-risk status instead of an optimistic schedule.

6.3 Future Work and Recommendations

The immediate next steps, in order, are:

1. Hold the interface-freeze review and ratify the SDD, record ADR-0001 and enable branch protection (the Stage 3 gate).
2. Implement the Raft core: node skeleton and timers, leader election, log replication and commit, with the in-process transport and cluster harness (Stage 3).
3. Connect real gRPC transport and the HTTP API, add leader redirection and `raftctl`, and run a three-node cluster on Docker Compose (Stage 4).
4. Add the write-ahead log, snapshots, compaction and crash-recovery tests (Stage 5).
5. Implement the join, learner and promotion protocol with membership tests (Stage 6).
6. Build the fault-injection suite, observability and the race-clean build (Stage 7), then finalise documentation, report, demonstration and presentation (Stage 8).

If time remains after the minimum viable product, the stretch goals are, in priority order: safe node removal, read-index reads to close the stale-read gap, a join token, extra metrics, chunked snapshot transfer and a small benchmark suite.

6.4 Final Remarks

RaftKV aims to show that Raft can be implemented correctly and understood in full by the team, and that a distributed system can be engineered with the discipline of a real software project: specification, interfaces, failure-oriented tests, automation and honest risk management. The work to date is the foundation; the evidence for the rest will be the tests, measurements and demonstration produced in the coming stages.

References

[1] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proc. USENIX Annual Technical Conference (USENIX ATC ’14)*, Philadelphia, PA, USA, 2014, pp. 305–319.

[2] D. Ongaro, “Consensus: Bridging Theory and Practice,” Ph.D. dissertation, Dept. of Computer Science, Stanford University, Stanford, CA, USA, 2014.

[3] L. Lamport, “Paxos Made Simple,” *ACM SIGACT News (Distributed Computing Column)*, vol. 32, no. 4, pp. 51–58, Dec. 2001.

[4] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.

Appendix A: HTTP API and CLI

All client traffic uses HTTP/JSON. Writes require the leader; a non-leader answers with the structured error of Section 3.4.4.

Table A.1: HTTP endpoints

Method	Path	Purpose	Leader only
PUT	`/kv/{key}`	Write the value of a key.	Yes (redirect)
GET	`/kv/{key}`	Read the value of a key.	Served by the leader in the mandatory scope
DELETE	`/kv/{key}`	Delete a key.	Yes (redirect)
GET	`/cluster/status`	Role, term, commit index and applied index of the node.	No
GET	`/cluster/leader`	Currently known leader.	No
GET	`/cluster/members`	Voter and learner list.	No
POST	`/cluster/join`	Request admission to the cluster as a learner (also available through the CLI).	Yes (redirect)
POST	`/cluster/remove`	Remove a member (SHOULD).	Yes (redirect)

`raftctl` is a thin, stateless wrapper over the HTTP API. It follows a leader hint once, printing a note such as “not leader, retrying against <leader>”.

```
raftctl put <key> <value>        raftctl cluster status
raftctl get <key>                raftctl cluster leader
raftctl delete <key>             raftctl cluster members
                                 raftctl cluster join <address>
                                 raftctl cluster remove <node-id>   # SHOULD
```

Appendix B: Phase Breakdown

The plan comprises 39 phases (epics) containing 161 tasks. The owner is the primary driver of the phase; windows are the planned dates of the phase's tasks.

Table B.1: Phases, owners, windows and effort

Phase	Title	Owner	Window	Tasks	Hours
1.1	Repository & Tooling Bootstrap	Person 4	Built; close-out S1	5	11
1.2	Core Interface Contracts (Storage, Transport, StateMachine)	Person 1	12 Oct – 14 Oct	6	16
1.3	Protobuf Contracts & gRPC Service Skeleton	Person 3	Built; close-out S1	5	14
1.4	HTTP Client API & raftctl CLI Skeletons	Person 3	Built; close-out S1	5	11
1.5	Docker Compose & Multi-Node Bootstrap Skeleton	Person 4	Built; close-out S1	5	10
1.6	CI Pipeline & Quality Gates	Person 4	12 Oct – 13 Oct	4	7
1.7	SDD Ratification, ADRs, Diagrams & Scope Guardrail	Person 4	12 Oct – 16 Oct	4	6
2.1	KV Store Core Data Structure & CRUD	Person 2	Built; close-out S1	4	10
2.2	Apply(command) Implementation & Determinism	Person 2	Built; close-out S1	4	7
2.3	Snapshot & Restore	Person 2	Built; close-out S1	5	11
2.4	Isolation Enforcement & Property Test Suite	Person 2	Built; close-out S1	5	14
2.5	Standalone Integration Stub (No-Raft Harness)	Person 2	Built; close-out S1	3	5
3.1	Raft Node Skeleton, State & Timers	Person 1	15 Oct – 23 Oct	5	22
3.2	Leader Election	Person 1	20 Oct – 30 Oct	5	31
3.3	Log Replication & Commit	Person 1	27 Oct – 06 Nov	6	42
3.4	In-Process Transport & Raft Test Cluster	Person 4	16 Oct – 06 Nov	5	21
4.1	Real gRPC Transport	Person 3	09 Nov – 20 Nov	4	21
4.2	Real HTTP API	Person 3	09 Nov – 20 Nov	4	18
4.3	Leader Redirect & raftctl	Person 3	16 Nov – 20 Nov	3	10
4.4	Multi-Node End-to-End on Docker Compose	Person 4	19 Nov – 27 Nov	4	17
5.1	Write-Ahead Log	Person 2	09 Nov – 20 Nov	5	31
5.2	Raft and Storage Integration	Person 2	16 Nov – 27 Nov	3	13
5.3	Snapshots & Log Compaction	Person 2	23 Nov – 04 Dec	5	27
5.4	Crash Recovery Validation	Person 2	30 Nov – 04 Dec	3	13
6.1	Config-Change Entries & Durable Membership	Person 2	23 Nov – 01 Dec	4	22
6.2	Join Protocol	Person 2	30 Nov – 07 Dec	4	18

Phase	Title	Owner	Window	Tasks	Hours
6.3	Learner Catch-Up & Promotion	Person 2	03 Dec – 10 Dec	4	15
6.4	Membership Test Suite	Person 4	08 Dec – 11 Dec	5	19
7.1	Fault Injection Harness	Person 4	02 Nov – 20 Nov	4	24
7.2	Failure Scenario Suite	Person 4	23 Nov – 11 Dec	6	25
7.3	Correctness Properties & Race-Clean Build	Person 4	30 Nov – 11 Dec	3	19
7.4	Observability (Structured Logs & Basic Metrics)	Person 3	26 Oct – 11 Dec	3	13
8.1	End-to-End Integration & Demo	Person 4	07 Dec – 16 Dec	3	13
8.2	Documentation	Person 3	07 Dec – 16 Dec	4	13
8.3	Evaluation Deliverables	Team	08 Dec – 17 Dec	3	23
8.4	Release & Retrospective	Person 4	16 Dec – 18 Dec	3	5
9.1	Read-Index Linearizable Reads	Person 1	Unscheduled	1	12
9.2	Safe Node Removal	Person 2	Unscheduled	3	15
9.3	Hardening & Polish	Person 3	Unscheduled	4	23

Appendix C: Reproducing the Current Build

The following commands build and run the current repository (Go 1.27.1 as specified in the repository README; Docker with Compose is required separately for the last block).

```
# build, vet, test and race-test everything
make build vet test race

# run a node (stub API: HTTP :8001, gRPC :9001) and query it
./bin/raftkv
./bin/raftctl status

# opt-in local demo console (no consensus, in-memory only)
./bin/raftkv --demo --http-addr 127.0.0.1:8080 --grpc-addr 127.0.0.1:9090
# then open http://127.0.0.1:8080/demo/

# three-node Compose skeleton; add a fourth and fifth process via profiles
docker compose up -d --build
docker compose --profile node4 up -d --build
docker compose --profile node4 --profile node5 down
```

Appendix D: Glossary

Table D.1: Terms used in this report

Term	Meaning
Term	A logical epoch in Raft; each term has at most one leader.
Quorum	A majority of voting members, ⌊N/2⌋ + 1; any two quorums overlap.
Leader, follower, candidate	The three Raft roles. Only the leader accepts writes.
Learner	A non-voting member that receives replication but does not count for quorum; used while a new node catches up.
Log entry	One replicated command with an index and the term in which it was created.
Commit index, applied index	Highest entry known to be on a quorum; highest entry applied to the state machine.
WAL	Write-ahead log: an append-only file in which state is recorded before it is acted upon.
fsync	System call that forces buffered file data to stable storage.
Snapshot	A serialised copy of the state machine, used to compact the log and to bring nodes up to date.
Split brain	Two parts of a cluster both believing they may accept writes, leading to divergent state.
Linearizable read	A read that returns the result of the latest completed write.
ADR	Architecture Decision Record: a short document capturing one design decision, its context and consequences.
MoSCoW	Prioritisation scheme: Must, Should, May (could), and out of scope.